



3 p.2
17
2550 sp

عملي

كلية الهندسة المعلوماتية
السنة الثانية

تمارين على ال BST

عملي مشترك



RB Informatics; 25/5/2026
محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

بسم الله الرحمن الرحيم

في المحاضرة السابقة تعرفنا على مفهوم الـ BST وبعض الخصائص الأساسية أما اليوم فسننتقل إلى التطبيق العملي من خلال مسائل متنوعة، وسنرى كيف يمكن استغلال خاصية الترتيب في الـ BST لتبسيط الخوارزميات وتحسين أدائها

التمارين الواردة في المحاضرة	
رقم الصفحة	عنوان التمرين
2	التحقق من صحة الـ BST
4	إثبات أن للشجرة نهاية ميتة (Dead End)
6	التحقق أن المصفوفة المعطاة ناتجة عن اجتياز الـ Inorder لـ BST
7	بناء الـ BST من اجتياز preorder
10	إيجاد أقل سلف مشترك (LCA)
14	حساب المسافة بين عقدتين

Exercise 1

1. ليكن لدينا شجرة ثنائية اكتب تابع يتأكد من كون هذه الشجرة هي شجرة بحث ثنائية:

فكرة الحل: سنعتمد على تمرير المجال المسموح لكل عقدة، بدلاً من مقارنة العقدة بأولادها فقط (وهو خطأ شائع لأن قد تنتهك العقدة شرط أحد الأجداد رغم التزامها بشرط أبيها)

الطريقة:

عند كل عقدة نتحقق أولاً من أن قيمتها تقع ضمن نطاقها المسموح. ($\min < \text{node.value} < \max$).

ثم نقوم بتعديل المجال للأبناء بناءً على قيمة العقدة الحالية كالتالي:

عند النزول إلى الابن الأيسر، نبقى الحد الأدنى $\min$ كما هو بينما نجعل الحد الأعلى $\max$ الجديد مساوياً لقيمة العقدة الحالية (لأن الابن الأيسر يجب أن يكون أصغر من أبيه)، فتصبح نطاقه ($\min, \text{node.value} - 1$).

أما عند النزول إلى الابن الأيمن، فنجعل الحد الأدنى $\min$ الجديد مساوياً لقيمة العقدة الحالية، ونبقى الحد الأعلى $\max$ كما هو (لأن الابن الأيمن يجب أن يكون أكبر من أبيه)، فيصبح نطاقه ($\text{node.value} + 1, \max$).

تستمر هذه العملية بشكل عودي على كامل الشجرة، فإذا خالفت أي عقدة شرط النطاق الخاص بها، نرد القيمة False.

إذا أتم الاجتياز لجميع العقد بنجاح كانت الشجرة BST وترد النتيجة true

ملاحظة: لا يمكن للقيمة التي نختبرها أن تساوي أطراف المجال وذلك لضمان عدم تكرار القيم ضمن الشجرة

■ #كيف نبدأ ؟

نبدأ من الجذر حيث النطاق المسموح لقيمه غير محدود:

$\max = +\infty$ و $\min = -\infty$

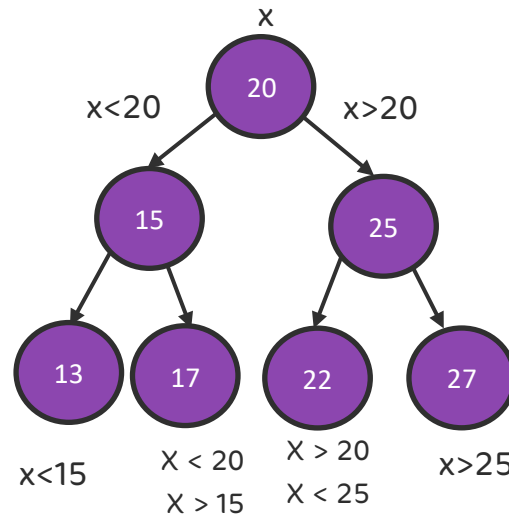
نتحقق هل قيمته بين $-\infty$ و $+\infty$ ؟ طبعاً نعم (لأي قيمة صحيحة تقع بينهما) ثم نتقل إلى اليسار مع مجال:

$(-\infty, \text{root.value} - 1)$

وإلى اليمين مع مجال:

$(\text{root.value} + 1, +\infty)$

وهكذا.



CODE

```

1 class BinaryTree {
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             this.left = this.right = null; }
9     public boolean isBST (Node root,int min , int max){
10         if (root==null)
11             return true;
12         if(root.value<=min || root.value>=max)
13             return false;
14         return isBST(root.left,min,root.value)&&isBST(root.right,root.value,max);}
15 }
16 public class Main {
17     public static void main(String[] args) {
18         BinaryTree bt = new BinaryTree();
19         bt.root = bt.new Node(20);
20         bt.root.right = bt.new Node(25);
21         bt.root.right.left = bt.new Node(22);
22         bt.root.right.right = bt.new Node(27);
23         bt.root.left= bt.new Node(15);
24         bt.root.left.right = bt.new Node(17);
25         bt.root.left.left = bt.new Node(13);
26         System.out.println(bt.isBST(bt.root,Integer.MIN_VALUE,Integer.MAX_VALUE)); }
27

```

Exercise 2

2. أثبت أن لشجرة البحث الثنائي نهاية ميتة (Dead End) ؟ تكون لشجرة البحث الثنائي نهاية ميتة إن كان في الشجرة عقدة ورقية لا يمكن إضافة أي عقدة جديدة تحتها (لا يساراً ولا يميناً) لأن المجال المتاح للقيمة الجديدة أصبح فارغاً (لا يحتوي على أي عدد صحيح)

فكرة الحل:

نستخدم تضيق المجال أثناء اجتياز الشجرة (كما في التمرين السابق)

الطريقة:

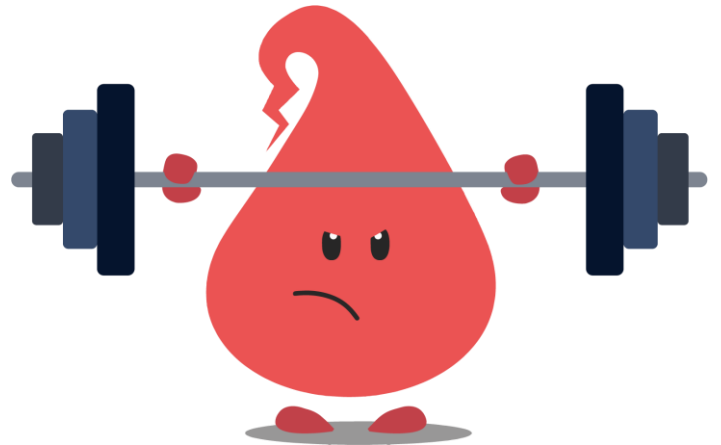
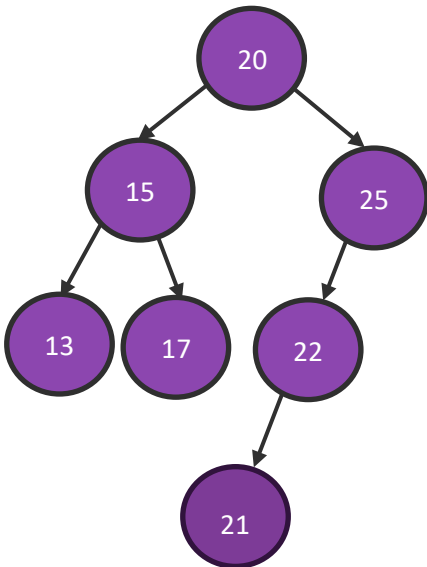
نبدأ للجذر بـ $\min = 1$ و $\max = \infty$ حيث نفترض أننا نخزن في الشجرة فقط قيماً صحيحة موجبة.

عند الانتقال إلى الابن الأيسر، نضيق المجال إلى $(\min, \text{root.value} - 1)$.

عند الانتقال إلى الابن الأيمن، نضيق المجال إلى $(\text{root.value} + 1, \max)$.

إذا وصلنا لعقدة (موجودة بالفعل) وكان المجال الذي أتيناها لها هو $\min == \max$ فهذا يعني أنه لا توجد قيمة واحدة صحيحة يمكن وضعها في هذا الفرع.

وبالتالي هذه العقدة تشكل نهاية ميتة





دعنا تتبع الفرع الأيمن من الشجرة لنرى كيف وصلنا إلى الـ Dead End:

عند العقدة: (20)

إذا أردنا التحرك يميناً، فإن أي عقدة قادمة يجب أن تكون أكبر من 20. المجال المتاح للعقد القادمة يصبح: $(21, +\infty)$:

عند العقدة: (25)

العقدة 25 تقع ضمن المجال السابق. الآن إذا أردنا التحرك يساراً من 25، يجب أن تكون العقدة أصغر من 25 وفي نفس الوقت أكبر من 20. المجال يضيق ليصبح: $(21, 24)$:

عند العقدة: (22)

العقدة 22 تقع تماماً بين 21 و 24 (مقبولة). الآن إذا أردنا التحرك يساراً من 22، يجب أن تكون العقدة القادمة أصغر من 22 ولكنها لازالت محكومة بالحد الأدنى السابق (أكبر من 20). المجال يضيق ليصبح: $(21, 21)$:

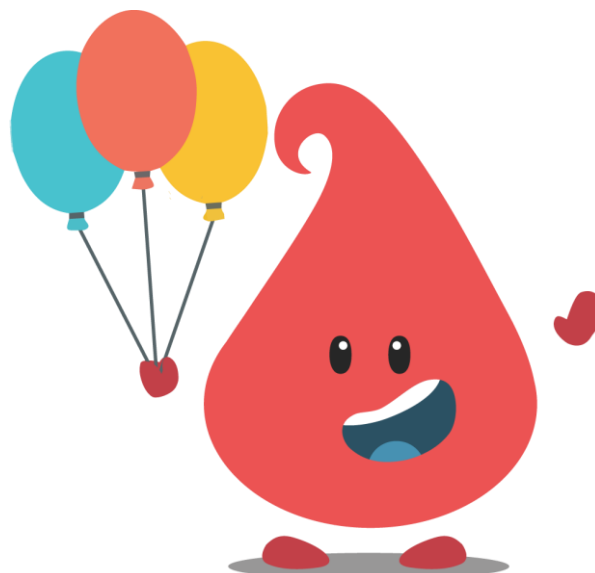
ملاحظة: المجال $(21, 21)$ يعني أن الرقم الوحيد المسموح به هنا هو 21 فقط.

عند العقدة: (21)

قمنا بإضافة العقدة 21 وهي صحيحة. الآن، لنحاول حساب المجال المتاح لأبناء العقدة 21: إذا أردنا الانتقال يساراً: يجب أن تكون القيمة أقل من 21 وأكبر من 20، فيصبح المجال المتاح $(21, 20)$.

هنا المشكلة! لا يوجد أي عدد صحيح يكون أكبر من 21 وأصغر من 20 في نفس الوقت

. $(\min > \max)$



Code

```

1 class BinaryTree {
2     Node root;
3     class Node {
4         int value;
5         Node left, right;
6         Node(int value) {
7             this.value = value;
8             this.left = this.right = null; }
9     public boolean hasDeadEnd (Node root,int min , int max){
10        if (root == null)
11            return false;
12        if (min == max)
13            return true;
14        return hasDeadEnd(root.left,min,root.value-1)||hasDeadEnd(root.right,root.value+1,max);}
15 }
16 public class Main {
17     public static void main(String[] args) {
18         BinaryTree bt = new BinaryTree();
19         bt.root = bt.new Node(20);
20         bt.root.right = bt.new Node(25);
21         bt.root.right.left = bt.new Node(22);
22         bt.root.right.left.left = bt.new Node(21);
23         bt.root.right.right = bt.new Node(27);
24         bt.root.left = bt.new Node(15);
25         bt.root.left.right = bt.new Node(17);
26         bt.root.left.left = bt.new Node(13);
27         System.out.println(bt.hasDeadEnd(bt.root,1,Integer.MAX_VALUE)); }

```

النتيجة التي سيطبعها الكود هي true :

عند تشغيل البرنامج، يقوم التابع hasDeadEnd بفحص الشجرة تراجعيًا: (Recursively)

في القسم الأيسر: يفحص العقد 15، 13، و 17 وينتهي بإعادة false

في القسم الأيمن: ينتقل الكود عبر المسار 20 \ 25 \ 22 وصولاً إلى العقدة 21.

min= 21, max = 21

وهنا يتحقق الشرط الأساسي داخل التابع:

min=max

يعيد التابع القيمة true فوراً ويتوقف البرنامج عن البحث.

EXERCISE 3

3. -ليكن لدينا مصفوفة أعداد صحيحة، اكتب تابع للتحقق أن المصفوفة المعطاة ناتجة عن اجتياز In-Order لشجرة بحث ثنائية (BST)

الفكرة:

إن اجتياز Inorder لأي شجرة BST يعطي قيماً مرتبة تصاعدياً. لذلك الشرط الوحيد الذي يجب تحققه هو أن المصفوفة غير متناقصة (متزايدة تماماً) (أي $arr[i] > arr[i-1]$ لكل i)

Code

```
1 public class Exercise3 {
2
3     public static boolean isInorderOfBST(int[] arr) {
4         if (arr == null || arr.length <= 1) {
5             return true;
6         }
7
8         for (int i = 1; i < arr.length; i++) {
9             if (arr[i] <= arr[i - 1]) {
10                 return false;
11             }
12         }
13         return true;
14     }
15
16     public static void main(String[] args) {
17         int[] arr1 = {1, 3, 4, 6, 7, 8, 10};
18         int[] arr2 = {1, 3, 6, 4, 7, 8};
19
20         System.out.println(isInorderOfBST(arr1)); // true
21         System.out.println(isInorderOfBST(arr2)); // false
22     }
23 }
24
```



- Input : arr[] = { 19, 23, 25, 30, 45 }
Output : Yes
- Input : arr[] = { 19, 23, 30, 25, 45 }
Output : No

Exercise 4

4. لديك مصفوفة تمثل ترتيب preorder لشجرة بحث ثنائية. قم ببناء الشجرة الأصلية وإرجاع جذورها.

فكرة الحل : لدينا في الـ BST لكل عقدة قيمة value جميع قيم الشجرة الفرعية اليسرى تقع في المجال (min, value) وجميع قيم الشجرة الفرعية اليمنى تقع في المجال (value, max) حيث min و max هما حدود النطاق المسموح به لتلك الشجرة الفرعية.

بما أن الـ preorder يزور الجذر أولاً ثم الشجرة الفرعية اليسرى كاملة ثم اليمنى كاملة، فإن أول عنصر في المصفوفة هو جذر الشجرة كلها، والعناصر التي تليها والتي تنتمي إلى النطاق (min, key) ستكون في الشجرة اليسرى، والعناصر التي تنتمي إلى النطاق (key, max) ستكون في الشجرة اليمنى.

خطوات الحل -1: لنستدعي الدالة التي تأخذ:

pre المصفوفة.

PreIndex (يمرر بالمرجع ليتقدم أثناء البناء).

Key قيمة العقدة الحالية (root أولاً)

Min ($-\infty$ أولاً)

Max ($+\infty$ أولاً)

size حجم المصفوفة

2- إذا كانت key لا تقع ضمن (min, max) نرجع null (هذا العنصر ليس من هذه الشجرة الفرعية)

3- ننشئ العقدة الحالية بقيمة key ونزيد المؤشر

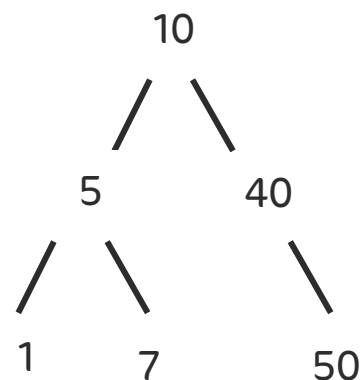
4- لبناء الشجرة اليسرى: نستدعي الدالة مع min نفسه و max = key وقيمة العقدة التالية في المصفوفة. جميع العقد التي تقع في هذا النطاق ستكون يساراً

5- لبناء الشجرة اليمنى: نستدعي الدالة مع min = key و max نفسه وقيمة العقدة التالية (بعد الانتهاء من اليسار).
6- . نعيد العقدة الحالية.

تعقيد الحل هو $O(n)$ حيث نمر على جميع عناصر المصفوفة مرة واحدة

Output:

Input : {10 , 5 , 1 , 7 , 40 , 50 }



```

1 class Index {
2   int index;
3   Index() { index = 0; }
4 }
5
6 public class Exercise4 {
7   Node root;
8
9   class Node {
10    int data;
11    Node left, right;
12    Node(int data) {
13      this.data = data;
14      left = right = null;
15    }
16  }
17
18  private Node constructBSTFromPreorder(int[] pre, Index preIndex, int key, int min,
19  int max, int size) {
20    // إذا انتهت المصفوفة أو القيمة خارج النطاق المسموح
21    if (preIndex.index >= size)
22      return null;
23
24    Node root = null;
25    if (key > min && key < max) {
26      root = new Node(key);
27      preIndex.index++; // ننتقل إلى العنصر التالي
  
```

```
28 // النطاق min .. key) بناء الشجرة اليسرى
29 if (preIndex.index < size) {
30     root.left = constructBSTFromPreorder(pre, preIndex, pre[preIndex.index],
31         min, key, size);
32 }
33 // النطاق key .. max) بناء الشجرة اليمنى
34 if (preIndex.index < size) {
35     root.right = constructBSTFromPreorder(pre, preIndex, pre[preIndex.index],
36         key, max, size);
37 }
38 }
39 return root;
40 }
41
42 public void inorder(Node root) {
43     if (root == null) return;
44     inorder(root.left);
45     System.out.print(root.data + " ");
46     inorder(root.right);
47 }
48
49 public static void main(String[] args) {
50     int[] pre = {10, 5, 1, 7, 40, 50};
51     Exercise4 b=new Exercise4();
52     Index preIndex = new Index();
53     Node root = b.constructBSTFromPreorder(pre, preIndex, pre[0],Integer.MIN_VALUE,
54     Integer.MAX_VALUE, pre.length);
55     System.out.print("Inorder traversal: ");
56     b.inorder(root); // 1 5 7 10 40 50
57 }
```

Exercise 5

5. - ليكن لدينا شجرة بحث ثنائي المطلوب

- أوجد السلف المشترك الأقرب لعقدتين a و b

LCA - (Lowest Common Ancestor).

السلف المشترك الأقرب لعقدتين a, b هو أقرب عقدة يلتقي عندها المسار من العقدة a حتى الجذر والمسار من العقدة b حتى الجذر.

ملاحظة: من المؤكد وجود سلف مشترك لعقدتين موجودتين في الشجرة، أي لا يوجد حالة لا يكون فيها سلف مشترك كون الجذر سلفاً لجميع العقد.

فكرة الحل: لدينا في الـ BST يمكن إيجاد الـ LCA بسهولة باستخدام خاصية الترتيب.

حيث نبدأ من الجذر، ونميز حالتين:

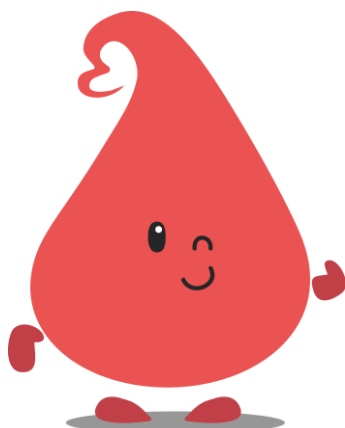
الحالة الأولى: a, b في جهة واحدة بالنسبة للعقدة الحالية، أي إما كلتا العقدتين أكبر من العقدة الحالية فنذهب يميناً أو كلتا العقدتين أصغر من العقدة الحالية فنذهب يساراً.

الحالة الثانية: a, b في جهتين مختلفتين بالنسبة للعقدة الحالية: أي أحدهما أصغر من العقدة الحالية والآخر أكبر أو أحدهما يساوي العقدة الحالية، وبالتالي العقدة الحالية هي نقطة الافتراق وهي الـ LCA

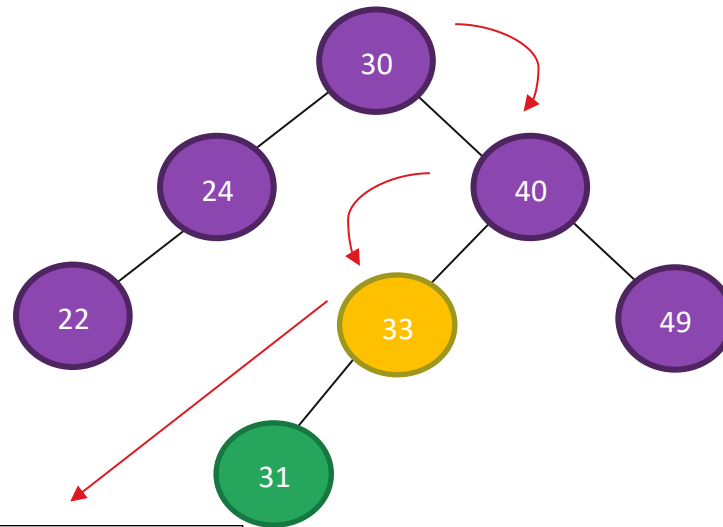
ملاحظة مهمة: نحن نفترض أن العقدتين a, b موجودتان في الشجرة

"ليس الحلم ما تراه في نومك ،

الحلم هو ما يمنعك من النوم"



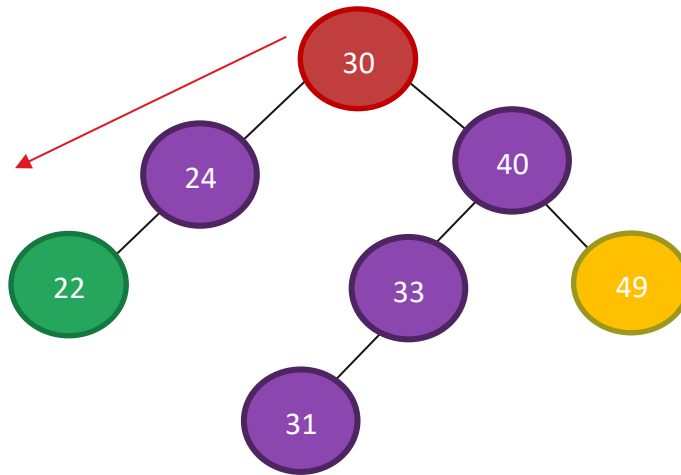
الحالة الأولى:



A=31
B=33

لم يعودا في نفس الجهة بالنسبة للعقدة
فتوقف الاستدعاء هنا
و تكون هذه العقدة هي السلف المشترك
(33) سواء كانت a أو b أو غير ذلك

الحالة الثانية:



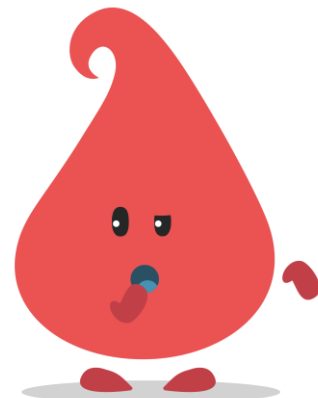
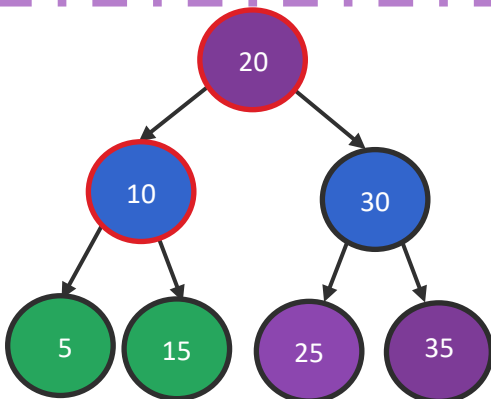
A=22
B=49

العقدتان ليستا في
نفس الجهة بالنسبة
للعقدة الحالية فتوقف
الاستدعاء هنا
و تكون هذه العقدة
هي السلف المشترك
(30)
سواء كانت a أو b أو
غير ذلك

```

2 class Exercise5 {
3     Node root;
4
5     class Node {
6         int data;
7         Node left, right;
8
9         Node(int data) {
10             this.data = data;
11             left = right = null;
12         }
13     }
14
15     private Node insert(Node root, int value) {
16         if (root == null) return new Node(value);
17         if (value < root.data)
18             root.left = insert(root.left, value);
19         else if (value > root.data)
20             root.right = insert(root.right, value);
21         return root;
22     }
23
24     // نفترض أن n1 و n2 موجودان
25     public Node lca(Node node, int n1, int n2) {
26         if (node == null) {
27             return null;
28         }
29
30         إذا كانت n1 و n2 اصغر من العقدة الحالية ← LCA في اليسار
31         if (node.data > n1 && node.data > n2) {
32             return lca(node.left, n1, n2);
33         }
34

```



```

35 // إذا كانت n1 و n2 اكبر من العقدة الحالية ← LCA في اليمين
36 if (node.data < n1 && node.data < n2) {
37     return lca(node.right, n1, n2);
38 }
39
40 // العقدة الحالية تفصل بين n1 و n2 او تساوي احدهما
41 return node;
42 }
43
44
45 private void inorder(Node root) {
46     if (root != null) {
47         inorder(root.left);
48         System.out.print(root.data + " ");
49         inorder(root.right);
50     }
51 }
52
53 public static void main(String[] args) {
54     Exercise5 tree = new Exercise5();
55     Node root = null;
56     int[] values = {20, 10, 30, 5, 15, 25, 35};
57     for (int v : values)
58         root = tree.insert(root, v);
59
60     System.out.print("Inorder traversal: ");
61     tree.inorder(root); // 5 10 15 20 25 30 35
62     System.out.println();
63
64     Node lca1 = tree.lca(root, 5, 15);
65     System.out.println("LCA(5,15) = " + lca1.data); // 10
66
67     Node lca2 = tree.lca(root, 10, 30);
68     System.out.println("LCA(10,30) = " + lca2.data); // 20
69
70 }
71 }

```

Exercise 6

6. لدينا شجرة بحث ثنائي (BST) اكتب تابع لحساب المسافة بين عقدتين a و b في الشجرة:

فكرة الحل: المسافة بين عقدتين في شجرة هي عدد الحواف (edges) في المسار الذي يربط بينهما. لحساب المسافة بين عقدتين في الـ BST نعتمد على فكرة السلف المشترك الأقرب LCA لعقدتين.

تتكون المسافة من جزأين: المسافة من العقدة الأولى إلى LCA ثم المسافة من LCA إلى العقدة الثانية

الخطوات:

إيجاد السلف المشترك الأقرب (LCA) للعقدتين المطلوبتين (كما في التمرين السابق).
حساب المسافة من LCA إلى كل عقدة عن طريق عدد الحواف أثناء النزول من LCA إلى العقدة المطلوبة.
المسافة الكلية = المسافة (LCA, node1) + المسافة (LCA, node2)

■ مثال للتوضيح

حساب المسافة بين 3 و 7:

(1) $LCA = 5$

(2) المسافة من 5 إلى 3 = 1

(3) المسافة من 5 إلى 7 = 1

(4) المسافة الكلية = 2

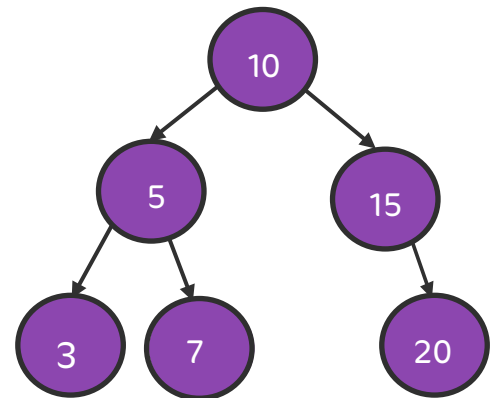
حساب المسافة بين 3 و 20:

(1) $LCA = 10$

(2) المسافة من 10 إلى 3 = 2

(3) المسافة من 10 إلى 20 = 2

(4) المسافة الكلية = 4



```
1 class Exercise6 {
2     Node root;
3
4     class Node {
5         int key;
6         Node left, right;
7
8         Node(int key) {
9             this.key = key;
10            left = right = null;
11        }
12    }
13
14    private Node insert(Node root, int key) {
15        if (root == null) return new Node(key);
16        if (key < root.key)
17            root.left = insert(root.left, key);
18        else if (key > root.key)
19            root.right = insert(root.right, key);
20        return root;
21    }
22    // حساب المسافة بين عقدتين
23    // تفترض أن a و b موجودان وأن a <= b
24    public int distanceBetween2(Node root, int a, int b) {
25        if (root == null)
26            return 0;
27        // كلاهما في اليسار
28        if (root.key > a && root.key > b)
29            return distanceBetween2(root.left, a, b);
30        // كلاهما في اليمين
31        if (root.key < a && root.key < b)
32            return distanceBetween2(root.right, a, b);
33        // root هو الـ LCA في جهتين مختلفتين و الـ a و الـ b
34        if (root.key >= a && root.key <= b)
35            return distanceFromRoot(root, a) + distanceFromRoot(root, b);
36        return 0;
37    }
38}
```

```

39  //x دالة لحساب المسافة من عقدة root الى القيمة x
40  public int distanceFromRoot(Node root, int x) {
41      if (root.key == x)
42          return 0;
43      else if (root.key > x)
44          return 1 + distanceFromRoot(root.left, x);
45      else
46          return 1 + distanceFromRoot(root.right, x);
47  }
48
49
50  private void inorder(Node root) {
51      if (root != null) {
52          inorder(root.left);
53          System.out.print(root.key + " ");
54          inorder(root.right);
55      }
56  }
57
58  public static void main(String[] args) {
59      Exercise6 tree = new Exercise6();
60
61      Node root=null;
62      int[] values = {20, 10, 30, 5, 15, 25, 35};
63      for (int v : values) {
64          root=tree.insert(root,v);
65      }
66
67      System.out.print("Inorder traversal: ");
68      tree.inorder(root); // 5 10 15 20 25 30 35
69      System.out.println();
70
71      System.out.println("Distance between 5 and 15: " +
72          tree.distanceBetween2(root, 5, 15)); // 2
73
74      System.out.println("Distance between 15 and 35: " +
75          tree.distanceBetween2(root, 15, 35)); // 4
76
77      System.out.println("Distance between 20 and 20: " +
78          tree.distanceBetween2(root, 20, 20)); // 0
79  }
80 }

```

